

Assignment 8

Creating a Progressive Web App

Anna's Kitchen Website Project

Project scenario

Anna's Kitchen now has a complete responsive website, but the client would like the site to provide a more app-like experience and remain accessible when the internet connection is unavailable. The client has asked for the website to be extended as a Progressive Web App (PWA) by adding application metadata and offline support. The existing design and content must remain unchanged.

Your task

The HTML structure and CSS styling for the Anna's Kitchen website have already been provided in the starter files. Your task is to extend the website with Progressive Web App features by adding a web app manifest and service worker.

Create the Web App Manifest

1. Create a new file named: `manifest.json`
2. Add the following properties to the manifest:
name: "Anna's Kitchen"
short_name: "AK"
description: "A client-side restaurant web-app."
start_url: `"/index.html"`
scope: `"/"`
display: "standalone"
background_color: `"#FFF8F0"`
theme_color: `"#8B1D2A"`
3. Add an icons array containing the two app icons:
`logo-192.png`
`logo-512.png`

Link the Manifest to the HTML Page

1. Open `index.html` and connect the webpage to `manifest.json`
2. The link should identify the file as the web app manifest.
3. Save the file and reload the webpage.

Create the Service Worker

1. Create a new JavaScript file named: `serviceWorker.js`
2. Create a cache name for the Anna's Kitchen application. For example: `annas-kitchen-v1`
3. Create a list containing the main website resources that should be stored in the cache:

```
./  
./index.html  
./styles.css  
./images/annas.jpeg
```

./images/map.png

4. Add an install event listener that opens the cache and stores the listed resources when the service worker is installed.
5. Add an activate event listener that checks the existing caches, removes any outdated caches, and allows the current service worker to take control of the application.
6. Add a fetch event listener that handles requests made by the website. The service worker should:
 - check whether the requested resource is already available in the cache
 - return the cached resource when one is available
 - retrieve the resource from the network when it is not cached
 - store successful network responses in the cache for future requests
 - return the cached index.html page if a navigation request fails because the network is unavailable

Register the Service Worker

1. Open index.html
2. Check whether the browser supports service workers.
3. If service workers are supported, register service-worker.js when the webpage has loaded.

Test the Application

1. Run the app using a local web server.
2. Open the browser developer tools and check that:
 - the web app manifest has been detected
 - the service worker has been registered and activated
 - the cache has been created
 - the specified resources have been stored in the cache
3. Load the website normally so that the required resources are cached.
4. Set the browser network connection to **Offline**.
5. Reload the webpage and confirm that the main page, CSS styling and cached images continue to load.

Supplied files

- assignment-8-starter-files.zip